

Playwright Grouping, Annotations, Hooks & Tagging

1. Test Grouping - `test.describe()`

Used to organize related test cases into logical groups.

Syntax

```
import { test, expect } from '@playwright/test';

test.describe('Login Tests', () => {

  test('Valid Login', async ({ page }) => {

  });

  test('Invalid Login', async ({ page }) => {

  });

});
```

Benefits

- Organizes tests
 - Improves readability
 - Allows group-level hooks
 - Apply annotations/tags to multiple tests
-

Nested Groups

```
test.describe('E-Commerce', () => {  
  
  test.describe('Login', () => {  
  
    });  
  
  test.describe('Cart', () => {  
  
    });  
  
});
```

2. Test Annotations

Annotations provide additional information about tests.

Skip Test

Skip a test completely.

```
test.skip('Feature not ready', async ({ page }) => {  
  
  });
```

Skip Conditionally

```
test.skip(process.platform === 'win32');
```

Only

Run only selected test(s).

```
test.only('Login Test', async ({ page }) => {  
  
  });
```

Note: Remove `test.only()` before committing code to version control.

Describe Only

```
test.describe.only('Smoke Tests', () => {  
  
  });
```

Runs only this test group.

Fixme

Marks a known broken or unfinished test.

```
test.fixme('Bug is under development');
```

Use when:

- Feature is broken
- Bug already reported
- Test should not execute

Fail

Expected failure.

```
test.fail('Known Bug');
```

If the test fails, Playwright treats it as expected.

Slow

Tripled the timeout.

```
test.slow();
```

Useful for:

- Large uploads
- Long-running workflows
- Slow environments

Annotation Summary

Annotation	Purpose
<code>test.skip()</code>	Skip test
<code>test.only()</code>	Run only selected test
<code>test.describe.only()</code>	Run only one group
<code>test.fixme()</code>	Known broken feature
<code>test.fail()</code>	Expected failure
<code>test.slow()</code>	Increase timeout

3. Hooks

Hooks execute before or after tests.

beforeAll()

Runs **once before all tests**.

```
test.beforeAll(async () => {  
  console.log('Runs once');  
});
```

Use for:

- Database connection

- Start server
 - Create test data
-

afterAll()

Runs **once after all tests**.

```
test.afterAll(async () => {  
  console.log('Runs once after all tests');  
});
```

Use for:

- Cleanup
 - Delete test data
 - Close connections
-

beforeEach()

Runs **before every test**.

```
test.beforeEach(async ({ page }) => {  
  await page.goto('https://example.com');  
});
```

Use for:

- Launch application
 - Login
 - Common setup
-

afterEach()

Runs **after every test**.

```
test.afterEach(async () => {  
    console.log('Test completed');  
});
```

Use for:

- Logout
- Cleanup
- Capture logs/screenshots

Hook Execution Order

```
beforeAll  
  
  beforeEach  
  Test 1  
  afterEach  
  
  beforeEach  
  Test 2  
  afterEach  
  
  beforeEach  
  Test 3  
  afterEach  
  
afterAll
```

4. Tagging Tests

Tags help categorize tests.

Single Tag

```
test('Login Test', {
  tag: '@smoke'
}, async ({ page }) => {

});
```

Multiple Tags

```
test('Checkout Test', {
  tag: ['@smoke', '@regression']
}, async ({ page }) => {

});
```

Tag Entire Group

```
test.describe('Login Module', {
  tag: '@login'
}, () => {

  test('Valid Login', async ({ page }) => {

  });

  test('Invalid Login', async ({ page }) => {

  });

});
```

All tests inherit the tag.

Running Tagged Tests

Run Smoke Tests

```
npx playwright test --grep @smoke
```

Run Regression Tests

```
npx playwright test --grep @regression
```

Run Login Tests

```
npx playwright test --grep @login
```

Exclude Smoke Tests

```
npx playwright test --grep-invert @smoke
```

Multiple Tags

Run Smoke **or** Regression

```
npx playwright test --grep "@smoke|@regression"
```

Run Smoke but Exclude Login

```
npx playwright test --grep @smoke --grep-invert @login
```

Example

```
import { test, expect } from '@playwright/test';

test.describe('Authentication', {
  tag: '@smoke'
}, () => {

  test.beforeEach(async ({ page }) => {
    await page.goto('https://example.com');
  });

  test('Valid Login', async ({ page }) => {

  });

  test.skip('Forgot Password', async ({ page }) => {

  });

  test.fixme('Social Login', async ({ page }) => {

  });

  test.fail('Known Login Bug', async ({ page }) => {

  });

});
```

Best Practices

- Group related tests using `test.describe()`.
- Keep setup code in `beforeEach()` and cleanup in `afterEach()`.
- Use `beforeAll()` and `afterAll()` only for expensive operations.
- Use `test.skip()` for features that are not ready.
- Use `test.fixme()` for known issues that should not run.
- Use `test.fail()` when a failure is expected.
- Use `test.slow()` only when a test genuinely needs more time.

- Use meaningful tags such as `@smoke`, `@regression`, `@sanity`, `@api`, and `@ui`.
- Remove `test.only()` before pushing code to the repository.

Quick Reference

Feature	Method
Group Tests	<code>test.describe()</code>
Before All Tests	<code>test.beforeAll()</code>
After All Tests	<code>test.afterAll()</code>
Before Each Test	<code>test.beforeEach()</code>
After Each Test	<code>test.afterEach()</code>
Skip Test	<code>test.skip()</code>
Run Only Test	<code>test.only()</code>
Run Only Group	<code>test.describe.only()</code>
Expected Failure	<code>test.fail()</code>
Broken Test	<code>test.fixme()</code>
Slow Test	<code>test.slow()</code>
Add Tags	<code>tag: '@smoke'</code>
Run Tagged Tests	<code>--grep @smoke</code>
Exclude Tagged Tests	<code>--grep-invert @smoke</code>

Beginner Tip: Think of `test.describe()` as a folder for related tests, **hooks** as setup/cleanup steps, **annotations** as special instructions for Playwright, and **tags** as labels that help you run only the tests you need.